EventHub | Three-person work division

Project: EventHub campus event booking on AWS

Region/account: Mumbai (ap-south-1), 061039790916

Prepared: 15 October 2026

Team names: Replace Member 1, Member 2, and Member 3 with your actual names before submission.

Shared goal

Demonstrate one complete booking from the website through SQS and Step Functions to DynamoDB and then show monitoring and audit evidence in the AWS Console. The browser must show the updated seat count after a confirmed booking, and a waitlisted booking must leave the seat count unchanged.

Member 1 — Cloud platform and security

Owns: VPC, public/private subnets, route tables, internet gateway, security groups, EC2/EBS, IAM roles, SSM Parameter Store.

- Verify the EC2 instance is healthy and the app and worker services are active.
- Show that the database accepts port 3306 only from the web security group and is not publicly accessible.
- Check that EC2 and Step Functions use project roles and that secrets are stored as SecureStrings.
- Keep the deployment command and rollback notes ready; coordinate the final app upload.

Console demonstration: VPC diagram and routes; EC2 instance and attached role; IAM role permissions; SSM parameter names and types. Hide secret values and use a disposable demo password.

Evidence: Screenshots of VPC/subnets/routes, security groups, EC2 health, IAM roles, and SSM parameter list.

Member 2 — Website, data, and booking experience

Owns: Flask website, MySQL on RDS, DynamoDB tables, signup/login, event list, seat-count display, My tickets page.

- Verify RDS is available and all twelve catalog events load in the website.
- Test the new processing state: after booking, the website waits for the workflow result and refreshes the seat count from DynamoDB.
- Demonstrate a confirmed booking, a zero-seat waitlist, and a duplicate click. Check that a confirmed booking deducts exactly one seat; waitlist and duplicate requests deduct none.
- Compare the website status with the EventSeats and Registrations table items.

Console demonstration: RDS configuration; DynamoDB EventSeats before/after; Registrations status; live website booking and My tickets.

Evidence: Screenshots of before/after seat counts, confirmed and waitlisted tickets, and the visible processing/result message.

Member 3 — Workflow, email, and operations

Owns: SQS and DLQ, Step Functions, SNS ticket and alarm topics, CloudWatch dashboard/alarms, CloudTrail, S3 trail bucket, budget evidence.

- Confirm the worker consumes the registration queue and the state machine follows its confirmed and waitlist branches.
- Verify student ticket subscriptions are confirmed and filtered to their own email; verify a real ticket email after filter propagation.
- Check alarms, dashboard, CloudTrail logging, S3 log destination, and the budget alert.
- Run a malformed-message test after the successful demo and show the DLQ/alarm; clear the test message afterward.

Console demonstration: SQS redrive settings; Step Functions execution graph; SNS subscriptions; CloudWatch dashboard and alarms; CloudTrail trail and S3 bucket; budget.

Evidence: Screenshots of a successful execution, filtered subscription, ticket email, DLQ test, alarm, trail status, and budget.

Joint rehearsal and handoff

1. Member 1 checks account 061039790916, Mumbai region, EC2/RDS health, security groups, and the deployed app version.
2. Member 2 creates disposable demo users and records the starting seat counts. Member 3 confirms the SNS subscriptions at least 15 minutes before the live email test.
3. Member 2 books a seat and shows the browser count change. Member 3 opens the same Step Functions execution and SNS email. All three compare the DynamoDB count and ticket status.
4. Test an event with zero seats and a duplicate request. Confirm no extra seat is deducted. Then test two available seats with three distinct users and show exactly two confirmed tickets and one waitlisted ticket.
5. Member 3 demonstrates the DLQ and alarm only after the successful transaction. Member 1 keeps the website available and records any failure logs.
6. Save screenshots with the resource name, account, region, date, and result. Complete the Actual column in docs/test-plan.md together.

Current handoff status

The EC2/RDS screenshots, HTTP 200 deployment check, integration script, CloudWatch alarms, and CloudTrail logging have been verified. The improved live seat-count display is in local code and still needs bash scripts/04-deploy.sh on the team Mac. A real student ticket email and operator alarm email still need inbox evidence.

Assessment guide: docs/console-walkthrough.md

Detailed test cases: docs/test-plan.md